

The `mm512` Header Library for AVX-512 Nested Multilevel Monte Carlo

Headers

`options_mm512.h`
`modes_mm512.h`

Target

Intel® Xeon® Gold 6538Y+
(Emerald Rapids)
AVX-512 FP16

Compiler

`icpx`, C++17

1 Introduction

This document describes the two headers that factor out everything the AVX-512 nested-MLMC estimators share. `options_mm512.h` is a reusable numerics library: Milstein steps, the Asian time-integral bridge, payoffs, correlated multi-asset draws and compensated summation, supplied as free functions carrying no global state. `modes_mm512.h` is the experiment scaffolding built on top of it: the three-mode precision sweep, its runtime switches, and its output policy.

The split is deliberate. The first header is a library; the second is one particular study. A user who wants only the numerics includes only the first, and drives `mlmc_test()` directly.

Table of Contents

2	Overview	3
3	Data Type Support	3
4	Overview of Helper Names	3
5	<code>options_mm512.h</code> — Numeric Instructions	3
5.1	Shared: Accumulation and Moments	3
5.2	<code>opt::scalar</code> — 1-Asset GBM	4
5.3	<code>opt::basket</code> — 5-Asset Correlated GBM	5
6	<code>modes_mm512.h</code> — The Sweep	5
6.1	Runtime Switches	5
6.2	The Three Modes	5
6.3	<code>SweepConfig</code>	6
6.4	<code>run_sweep</code>	6
7	Using the Headers	6
7.1	Include Order	6
7.2	Writing a New Payoff	6
7.3	Build	7

Tables

Table 1.	Supported vector types and lane counts
Table 2.	Naming convention for the helper functions
Table 3.	<code>opt</code> — shared accumulation and moment instructions
Table 4.	The <code>sums[]</code> moment vector
Table 5.	<code>opt::scalar</code> — 1-asset GBM instructions
Table 6.	<code>opt::basket</code> — 5-asset correlated GBM instructions
Table 7.	Runtime mode switches
Table 8.	The three sweep modes
Table 9.	<code>SweepConfig</code> fields

2 Overview

The two headers sit between the problem-agnostic MLMC harness and the payoff implementations:

```

./core/nested_mlmc_test.cpp  harness: convergence table, regression, complexity test
    ↑
modes_mm512.h               sweep scaffolding: modes, switches, run_sweep()
    ↑
options_mm512.h             numerics: Milstein, Asian bridge, payoffs, Kahan
    ↑
avx512_rng.h                Philox + inverse-CDF transforms

```

Everything is `static inline` in headers, consistent with this project’s single-translation-unit build: each estimator is one `.cpp` compiled directly, with no separate link step.

3 Data Type Support

Table 1. Supported vector types and lane counts

Type	Element	Notes
<code>_m512</code>	$16 \times \text{FP32}$	512-bit register. The FP32 path state and every dW draw.
<code>_m256h</code>	$16 \times \text{FP16}$	256-bit register. The FP16 path state.
<code>_Float16</code>	$1 \times \text{FP16}$	Scalar half, used for per-level constants and payoff evaluation.
<code>double</code>	$1 \times \text{FP64}$	The <code>sums[]</code> moment vector, always, regardless of path precision.

4 Overview of Helper Names

The helpers follow a fixed convention, so the precision and family of a call are legible without consulting the declaration:

`opt::family::opname width (arguments)`

`family` is `scalar` or `basket`; `width` is 32 for a `_m512` FP32 operation and 16 for a `_m256h` FP16 one. Every FP32 helper has an FP16 counterpart with an identical role.

Table 2. Naming Convention for the Helper Functions

Name	Description
<code>opt::scalar::mil_incr32</code>	One Milstein increment on 16 FP32 lanes, 1-asset.
<code>opt::scalar::mil_incr16</code>	The same on 16 FP16 lanes.
<code>opt::basket::step_pair32</code>	One coupled coarse step across 5 assets, FP32.
<code>opt::basket::step_pair16</code>	The same in FP16, routing state through an accumulator.
<code>opt::kahan_accum16</code>	Compensated FP16 add; shared by both families.

5 options_mm512.h — Numeric Instructions

Namespace `opt`, with two sub-namespaces: `opt::scalar` (1-asset GBM; option 1 = Asian) and `opt::basket` (5-asset correlated GBM; option 1 = European, 2 = Asian).

5.1 Shared: Accumulation and Moments

Table 3. `opt` — Shared Accumulation and Moment Instructions

Name	Description
<code>kahan_accum16</code>	<code>void(_m256h &sum, _m256h &comp, _m256h term)</code> Classical three-line Kahan compensated add. <code>comp</code> carries what the previous add rounded away and must persist across steps alongside <code>sum</code> .
<code>plain_accum16</code>	<code>void(_m256h &sum, _m256h &, _m256h term)</code> Uncompensated counterpart. The signature is deliberately identical, including the unused reference, so a caller swaps one for the other without altering the call site.

Name	Description
narrow16	<code>_m256h(_m512 x)</code> FP32 $\rightarrow$ FP16 across 16 lanes.
widen32	<code>_m512(_m256h x)</code> FP16 $\rightarrow$ FP32 across 16 lanes.
Moments::add	<code>void(double dP, double Pfv, double cost)</code> Accumulate one path into the thread-local moments. <code>dP</code> is the MLMC correction Y , <code>Pfv</code> the fine-level payoff, <code>cost</code> the timestep count.
Moments::flush	<code>void(double *sums) const</code> Add the block's seven moments into the shared <code>sums[]</code> . Called once per 16-lane block, at the end.

Table 4. The `sums[]` Moment Vector

Index	Quantity	Used for
<code>sums[0]</code>	$\sum C$	Cost per level; gives γ .
<code>sums[1]</code>	$\sum Y$	Mean correction; gives α .
<code>sums[2]</code>	$\sum Y^2$	Variance; gives β and the driver's sample allocation.
<code>sums[3]</code>	$\sum Y^3$	Kurtosis diagnostic.
<code>sums[4]</code>	$\sum Y^4$	Kurtosis diagnostic.
<code>sums[5]</code>	$\sum P_f$	Telescoping-sum consistency check.
<code>sums[6]</code>	$\sum P_f^2$	Telescoping-sum consistency check.

Why the OpenMP pragma is not in the header. Every estimator runs the same pattern: a parallel loop over 16-lane blocks, each accumulating thread-local moments flushed at the end. The bookkeeping (`Moments`) is shared, but the pragma itself cannot be, because an OpenMP directive binds to the loop textually following it and the loop bodies differ per payoff. The estimators therefore keep:

```
#pragma omp parallel for schedule(static) reduction(+:sums[0:7])
```

5.2 `opt::scalar` — 1-Asset GBM

The SDE is $dX = rX dt + \sigma X dW$, integrated with

$$\Delta X = rXh + \sigma X dW + \frac{1}{2}\sigma^2 X (dW^2 - h).$$

Table 5. `opt::scalar` — 1-Asset GBM Instructions

Name	Description
C32, C16	<code>struct { r, sig, halfsig2, hf, sqhf; }</code> Per-level broadcast coefficient blocks. C16 omits <code>sqhf</code> . Built once per level, not per step.
make32	<code>C32(float r, float sig, float h)</code> Build the FP32 coefficient block.
make16	<code>C16(float r, float sig, _Float16 h)</code> FP16 counterpart.
mil_incr32	<code>_m512(_m512 X, _m512 dW, const C32 &c)</code> One Milstein increment. Returns ΔX ; does not step the state, so the caller may route it through an accumulator.
mil_incr16	<code>_m256h(_m256h X, _m256h dW, const C16 &c)</code> FP16 counterpart.
Pair32	<code>struct { dW0, dW1, dI0, dI1; }</code> The four draws one coupled coarse step consumes.
step_pair32	<code>void(Xf, Xc, Af, Ac, const Pair32 &d, sig, hf, hc, q25hc, cf, cc)</code> One coupled coarse step: two fine steps on dW_0, dW_1 and one coarse step on their sum. Updates all four state variables in place.
Pair16	<code>struct { dW0, dW1, dI0, dI1; }</code> FP16 draw bundle.
step_pair16	<code>template<AccFn> void(Xf, Xc, Af, Ac, Xf_c, Xc_c, Af_c, Ac_c, const Pair16 &d, AccFn acc, sig, hf, hc, q25hc, cf, cc)</code> FP16 counterpart. Carries the four compensation terms and routes every state update through <code>acc</code> , which the caller sets to <code>kahan_accum16</code> or <code>plain_accum16</code> .
pay32	<code>double(int o, float K, float A)</code> $\max(0, A - K)$, the Asian call.
pay16	<code>double(int o, _Float16 K, _Float16 A)</code> FP16 counterpart; the comparison is performed in FP16, then widened.

5.3 opt::basket — 5-Asset Correlated GBM

Five assets, $NA = 5$, equicorrelation ρ factored as $\Sigma = LL^T$. Each asset carries its own σ_i ; the basket value is $\sum_i w_i x_i$.

Table 6. opt::basket — 5-Asset Correlated GBM Instructions

Name	Description
NA	static const int NA = 5 Number of assets. Every per-asset array is this long.
cholesky	void(float rho, float L[NA][NA]) Lower-triangular Cholesky factor of the equicorrelation matrix; upper triangle zeroed. Call once at start-up, typically via SweepConfig::setup.
draw_corr32	template<Rng,DrawFn> void(Rng &g, DrawFn draw, float scale, const float L[NA][NA], __m512 dw[NA]) Produces NA correlated increments: NA independent Normals from draw, Cholesky-combined, then scaled. The transform is a template parameter, keeping the header RNG-agnostic.
draw_corr16	template<Rng,DrawFn> void(Rng &g, DrawFn draw, _Float16 scale, const float L[NA][NA], __m256h dw[NA]) FP16 counterpart.
value32	__m512(const __m512 x[NA], const float w[NA]) Weighted basket value $\sum_i w_i x_i$.
value16	__m256h(const __m256h x[NA], const float w[NA]) FP16 counterpart.
wdot32	__m512(const a[NA], const b[NA], const float w[NA]) Weighted dot $\sum_i w_i a_i b_i$ — the Asian time-average bridge term.
wdot16	__m256h(const a[NA], const b[NA], const float w[NA]) FP16 counterpart.
step32	void(__m512 x[NA], const dw[NA], __m512 h, float r, const float sig[NA], __m512 sigx[NA]) Per-asset Milstein step, in place. Returns $\sigma_i x_i$ taken pre-step in sigx, which the Asian bridge requires.
incr16	void(const x0[NA], const dw[NA], __m256h h, float r, const float sig[NA], __m256h out[NA], __m256h sigx[NA]) FP16 counterpart returning the increment rather than stepping, so the caller can route it through an accumulator.
step_pair32	void(xf[NA], xc[NA], &Af, &Ac, dw0, dw1, di0, di1, hf, hc, q25hc, r, sig, w, bool asian) One coupled coarse step across all assets. The Asian time-integral terms are computed only when asian is true; for European they are dead.
step_pair16	template<AccFn,AccVecFn> void(..., xf_c[NA], xc_c[NA], &Af_c, &Ac_c, ..., AccFn acc, AccVecFn accv, ...) FP16 counterpart. acc accumulates the scalar running sums, accv the per-asset state arrays.
pay32	double(int o, float K, float B, float A) $o = 1$: $\max(0, B - K)$, European. $o = 2$: $\max(0, A - K)$, Asian.
pay16	double(int o, _Float16 K, _Float16 B, _Float16 A) FP16 counterpart.

6 modes_mm512.h — The Sweep

This header is the experiment, not the library. It supplies three runtime switches, a configuration struct, and one driver function.

6.1 Runtime Switches

Table 7. Runtime Mode Switches

Switch	Type	Meaning
option	extern int	1 or 2 — which payoff of the family. Defined by the estimator's own file, so each template names its own options.
kahan_mode	static bool	Compensate the running state: path position and running sums.
adaptive_mode	static bool	Enable the l_star cutoff. Only ever paired with kahan_mode = true.
l_star	static int	Grid level at and above which the pure-FP32 branch takes over.

The level estimator reads these; run_sweep() sets them.

6.2 The Three Modes

Table 8. The Three Sweep Modes

Mode	Kahan	Adaptive	Behaviour
nokahan	no	no	Plain +=. Convergence table only: β is often non-positive here, and the adaptive complexity sampler never returns against a non-positive β , so Eps[] is replaced by a zero terminator.

Mode	Kahan	Adaptive	Behaviour
kahan	yes	no	Compensated accumulation on both path position and running sums, in the FP16 chain and its FP32 twin alike. β becomes genuinely positive and the real complexity test completes.
adaptive	yes	yes	Kahan below grid level <code>l_star</code> ; pure FP32 at and above it. Above the cutoff there is no FP16 chain for an odd super-level to differ from, so the odd levels are returned empty and the scheme is standard non-nested PF32 MLMC there — one correction $P_k - P_{k-1}$ per grid level, carried by the even level.

6.3 SweepConfig

Table 9. SweepConfig Fields

Field	Type	Default	Meaning
prefix	const char*	""	Output filename stem.
family	const char*	""	Family name, shown in the banner.
opt_name	const char*[2]	"1", "2"	Display names for options 1 and 2.
n_opt	int	2	Payoffs this program declares. Set to 1 for a single-payoff build, so <code>run_sweep</code> never loops past option 1.
estimator	void(*)(int,int,double*)	---	The level estimator.
eps	const float*	null	Zero-terminated accuracy targets.
l_star	int[3]	0	Indexed by option; element 0 unused.
setup	void(*)()	null	One-off initialisation, e.g. Cholesky. May be null.
N	int	20000	Samples for the convergence table.
L	int	10	Levels for the convergence table.
NO	int	200	Initial samples per level in the complexity test.
Lmin	int	2	Minimum refinement level.
Lmax	int	20	Maximum refinement level.

6.4 run_sweep

```
static inline int run_sweep(const SweepConfig &cfg, int argc, char **argv);
```

Runs three modes $\times$ `cfg.n_opt` options, or a single option when `--option N` is given on the command line. For each combination it sets the switches, opens `<prefix>-<mode>-<option>.txt`, prints a banner, and calls `mlmc_test()` — with the caller's `Eps[]` in the Kahan modes and a zero terminator otherwise. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`, so `main()` can simply return it.

Pass `--option` on any multi-payoff build. A `main()` that declares more than one payoff loops over all of them by default, and a combined wall-clock silently averages payoffs that do not cost the same per sample — different `l_star`, different sample allocation — so timing must be reported per payoff. The scalar family now declares a single payoff (`n_opt` = 1, Asian), so its sweeps are per-payoff by construction; the basket family still carries two, and must be timed one option at a time.

7 Using the Headers

7.1 Include Order

```
#include "../core/nested_mlmc_test.cpp" // harness -- must come first
#include "avx512_rng.h" // Philox + inverse-CDF transforms
#include "options_mm512.h" // numerics
#include "modes_mm512.h" // sweep scaffolding
```

`options_mm512.h` requires `immintrin.h` for `_m512`, `_m256h` and `_Float16`. `modes_mm512.h` requires the harness, since `run_sweep()` calls `mlmc_test()`; the wrong order is a hard `#error` rather than a confusing missing-declaration message from deep inside the function.

7.2 Writing a New Payoff

1. Alias the family and define `option`:

```
namespace S = opt::scalar; // or opt::basket
int option; // 1 or 2
```

2. Write the level estimator against the standard `mlmc_l` contract. Read `kahan_mode`, `adaptive_mode`, `l_star` and `option` to decide what to compute; accumulate with `opt::Moments` and `flush()` into `sums[]`:

```
void my_l(int l, int N, double *sums);
```

3. Fill in a `SweepConfig` and hand it to `run_sweep`:

```
int main(int argc, char **argv)
{
    static const float EPS[] = { 0.01f, 0.02f, 0.05f, 0.1f, 0.2f, 0.0f };

    SweepConfig cfg;
    cfg.prefix = "nested_myopt_fp16_avx512";
    cfg.family = "myopt";
    cfg.opt_name[0] = "Call";
    cfg.opt_name[1] = "Put";
    cfg.estimator = my_l;
    cfg.eps = EPS;
    cfg.l_star[1] = 5;
    cfg.l_star[2] = 6;
    return run_sweep(cfg, argc, argv);
}
```

Choosing `l_star`. Derive it from the file's own no-Kahan data: the last grid level before the FP16 variance stops decaying and begins to climb. Values tuned against a different FP16 chain do not transfer reliably — the basket templates inherited `l_star = 6` from an earlier design without re-deriving it, and that inheritance is a known loose end rather than a validated choice.

Using the numerics alone. A user who wants the Milstein and payoff helpers but not this three-mode study includes only `options_mm512.h` and calls `mlmc_test()` directly. That is the reason the two headers are separate.

7.3 Build

```
source /opt/intel/oneapi/setvars.sh
icpx -O3 -march=native -std=c++17 \
--gcc-install-dir=/usr/lib/gcc/x86_64-linux-gnu/13 \
nested_scalar_milstein_fp16_avx512.cpp -o nsh_avx
```

Add `-qopenmp` for the threaded build. `setvars.sh` must be sourced before running an OpenMP binary as well as before building it, otherwise the loader fails on `libiomp5.so`.